

Understanding Guide — defend every line in Q&A;

North vs South India Regional Classifier · Team Hmmmmmmmmmm · Read this before you stand up

How to use this document. It is structured as five sections: (1) where the model lives and how to load it; (2) every technical term we used, defined in one sentence; (3) the training procedure with formulas; (4) the validation procedure (six stress tests, explained); (5) a Q&A; cheat sheet with the ten most likely questions and the answer for each. Read sections 1, 5, and then skim 2-4 if you have time.

1. Where the model lives (criterion: 'model + technical rigor')

Three trained boosters are saved on disk and in the GitHub repo. Open any of them with the snippet below.

File	Type	What it answers	AUC
models/region_classifier_v1.txt	LightGBM (binary)	Behaviour-only, leak-free — can behaviour tell if from S?	0.508
models/region_classifier_v2_structural.txt	LightGBM (binary)	Behaviour + distance_km — does urban distance make a difference?	0.966
models/lgbm_v1.txt	LightGBM (regression)	Original ETA model from morning (Time taken in minutes)	0.905
models/lgbm_final.txt	LightGBM (regression)	Final ETA model after pivot merge (if predicted Delta > 0)	0.914

1.1 How to load + use the model (live demo for judges)

Paste this in any Python prompt:

```
import lightgbm as lgb
booster = lgb.Booster(model_file='models/region_classifier_v2_structural.txt')
print(booster.num_trees()) # number of decision trees in the ensemble
print(booster.feature_name()) # column order required at prediction time

# Predict on new rows
import pandas as pd
from src.data import load_clean
from src.features import add_features
df = add_features(load_clean('data/raw/Zomato Dataset.csv'))
X = df[booster.feature_name()]
pred = booster.predict(X) # probability of being SOUTH
labels = (pred >= 0.5).astype(int) # 1=South, 0=North
```

To show the judges the model exists: open File Explorer → C:\Users\Rayyan Shaikh\Desktop\Zomato-delivery-estimation\models\. They'll see region_classifier_v2_structural.txt (size ~ 1.5 MB). On GitHub: navigate to models/ in the repo — same files visible. If a judge asks 'how big', say it's a 1.5 MB text-encoded gradient-boosted ensemble of ~600 trees with depth 6.

2. Glossary — every term we used, one sentence each

If a judge asks 'what is X?' use the one-sentence answer below. Don't recite a textbook.

AUC (ROC-AUC) — Area under the receiver-operating-characteristic curve — the probability that a randomly chosen positive sample ranks higher than a randomly chosen negative. 0.5 = random, 1.0 = perfect.

F1 score — Harmonic mean of precision and recall. Useful when classes are imbalanced. Range 0–1.

Balanced accuracy — Average of recall on each class — accuracy that is not skewed by class imbalance.

Confusion matrix — 2×2 table of TP, FP, FN, TN at a chosen probability threshold (we use 0.5).

LightGBM — Gradient-boosted decision tree library by Microsoft. Histogram-based, leaf-wise growth, fast on tabular data. Handles missing values and categoricals natively.

Gradient boosting — Ensemble that adds one decision tree at a time, each fitting the residual of the previous trees, weighted by a learning rate.

Leaf-wise vs level-wise growth — Leaf-wise grows the most impactful leaf first; level-wise grows every leaf at depth d before depth $d+1$. LightGBM uses leaf-wise (faster, slightly more prone to overfit, controlled by `num_leaves`).

CatBoost — A gradient-boosted tree library by Yandex with strong native handling of categorical features — used as a cross-check that our LGBM result is not framework-specific.

SHAP (TreeExplainer) — SHapley Additive exPlanations — a model-agnostic way to attribute each prediction to each feature. For trees, exact and fast. We use the mean absolute SHAP value per feature as its importance.

Class weight (balanced) — Re-weights samples inversely to class frequency so a 39/61 class split doesn't bias the loss. Equivalent to oversampling the minority class implicitly.

Early stopping — Stop adding trees when validation AUC stops improving for N rounds. Prevents overfit, makes the model size adapt to data.

KFold cross-validation — Split rows into K folds; train on $K-1$, test on the 1 held-out; rotate. We use $K=5$ with `random_state=42` everywhere.

StratifiedKFold — KFold that preserves the South / North ratio in every fold (39 / 61). Avoids accidentally putting all-South in one fold.

GroupKFold — KFold variant where rows from the same group (here: same city code) are never split across train/test. Used to test if the model generalises to UNSEEN cities, not just unseen orders from known cities.

Out-of-fold (OOF) predictions — For each row, the prediction made when that row was in the validation fold. Concatenating all OOF predictions gives one honest score per row.

Mann-Whitney U test — Non-parametric test for whether two distributions have the same median. We use it on every numeric feature to compare South vs North.

Chi-square test — Tests whether the joint distribution of two categorical variables differs from the product of their marginals. We use it on weather, traffic, vehicle, etc., crossed with region.

Kolmogorov-Smirnov (KS) test — Non-parametric test comparing two distributions in their entirety — picks up shape differences that Mann-Whitney (which only looks at medians) can miss. Statistic D = max distance between the two empirical CDFs.

Cramér's V — Effect-size measure for Chi-square — V near 0 = independence, $V = 1$ = perfect dependence. Reported because p-values alone can be 'significant' with no real magnitude.

Haversine distance — Great-circle distance between two points on a sphere given lat/lon, in km. Formula on the next page.

Spearman correlation (ρ) — Pearson correlation on RANKS instead of raw values. Robust to outliers and non-linearity. $\rho = 0.32$ between distance_km and Time_taken means a moderate monotonic relationship.

Native categorical handling — Passing a categorical column with its raw values (no one-hot encoding). LightGBM and CatBoost both find optimal splits over category subsets natively.

Data leak / temporal leak — When a feature contains information that wouldn't be available at prediction time, or that is a proxy for the label. Our case: month is a proxy for region because the data-collection window opened in different months per region.

Ablation study — Train the model with one feature added at a time and watch how the metric changes. The feature whose removal hurts most carries the signal.

Confusion matrix at threshold 0.5 — We say the prediction is South if predicted probability ≥ 0.5 ; below = North. TP/FP/FN/TN are then row counts of (true $\times$ predicted).

3. Training procedure — the exact recipe

3.1 Region label

For each row:

```
city_code = regex_extract('^[A-Z]+)RES', Delivery_person_ID)
state = CITY_CODE_MAP[city_code].state
region = 'South' if state in {Kerala, Tamil Nadu, Andhra Pradesh, Telangana, Karnataka}
else 'North'
is_south = 1 if region == 'South' else 0
```

3.2 Haversine distance (the structural feature)

```
R = 6371 km
Δφ = lat■ - lat■, Δλ = lon■ - lon■ (in radians)
a = sin²(Δφ/2) + cos(lat■) · cos(lat■) · sin²(Δλ/2)
d = 2R · arcsin(√a)
```

This produces distance_km — an order-level scalar in [0.5, 30] after our physical cap.

3.3 LightGBM classifier (Model C, structural)

```
objective = binary (logistic loss)
metric = AUC
n_estimators = 600 (with early_stopping_rounds = 40)
learning_rate = 0.04
num_leaves = 63
min_child_samples = 20
class_weight = 'balanced'
categorical_feature = ['Weather_conditions', 'Road_traffic_density',
'Type_of_vehicle', 'Type_of_order', 'City', 'Festival']
random_state = 42
```

Loss per row: $L = -[y \log(p) + (1-y) \log(1-p)]$ where $p = \sigma(\text{model output})$ and σ is the sigmoid.

3.4 Cross-validation

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_idx, valid_idx in skf.split(X, y):
    model = lgb.LGBMClassifier(**params)
    model.fit(X[train_idx], y[train_idx],
    eval_set=[(X[valid_idx], y[valid_idx])],
    callbacks=[lgb.early_stopping(40)])
    proba = model.predict_proba(X[valid_idx])[:, 1]
```

4. Validation procedure — six stress tests

Test	What it checks	Headline result
KS shape on every numeric	Distributions differ in SHAPE, even when not	Only distance_km significant (D=0.088, p≈10 ⁻¹⁰ ■■■■)
Dispersion ratio	South/North std on each feature ≈ 1.0?	All 12 features within 0.98 – 1.04. No spread difference.
Interaction ablation	Five engineered crosses; do any beat the baseline?	Only distance_km-derived features lift; rest add zero.
GroupKFold by city	Generalisation to unseen cities (not just un-	Behaviour only stays 0.506; structural reaches 0.911
Per-South-state 4-way	Is South India internally homogeneous?	Acc = 0.377 vs 0.250 random — weakly heterogeneous
CatBoost cross-check	Is the result an LGBM-only artefact?	AUC = 0.507 ≈ LGBM. No artefact.

5. Q&A; cheat sheet — ten questions the judges will ask

Q1. Where is your model?

models/region_classifier_v2_structural.txt — a 1.5 MB LightGBM booster, ~600 trees. Also available on GitHub at github.com/TheClazer/Zomato-delivery-estimation/blob/main/models/region_classifier_v2_structural.txt. We can demo loading it live: `lgb.Booster(model_file=...)`.

Q2. How accurate is it?

It depends on what we feed it. With raw coords (Model A): AUC = 1.000, by construction. With pure behaviour features (Model B'): AUC = 0.508, essentially random. With behaviour + the haversine pickup-drop distance (Model C, structural): AUC = 0.996 (StratKFold) and 0.911 when we hold out entire cities (GroupKFold). All three are reported because each answers a different operational question.

Q3. Why three models, not one?

Each model isolates a different layer of regional signal. Model A confirms the labels are clean; Model B' tells us behaviour is uniform; Model C tells us geometry differs. Reporting only Model C would hide the fact that the lift comes entirely from one feature.

Q4. Why did you drop the 'month' feature?

Within-region month percentages differ even though calendar date ranges are identical (Feb 11 – Apr 6 2022 for both regions). February is 20.8% of North orders but only 4.3% of South — a data-collection-window artefact, not seasonality. Including it inflates the AUC of Model B from 0.51 to 0.59 by acting as a region proxy.

Q5. Could the result be a LightGBM quirk?

We ran the same task with CatBoost (Yandex's gradient-boosting library) — got AUC 0.507 vs LightGBM's 0.508 on the leak-free behaviour model. Cross-framework confirmation.

Q6. How do you know distance is a structural and not a behaviour feature?

Distance is computed as haversine of pickup and drop coordinates — it's a function of the city's geometric layout (where restaurants sit relative to customers), not how riders or customers behave. The KS test shows the distribution shape differs by region ($D = 0.088$, $p \approx 10^{-10}$) even when the medians and IQRs are nearly identical.

Q7. Why did Fold 4 of GroupKFold drop to 0.642?

That fold held both Kochi (Kerala) and Hyderabad (Telangana) at the same time. Our remaining South training cities were all Karnataka and Tamil Nadu — the model couldn't fully generalise to Kerala-and-Telangana geometry. It's a real, honest limit: South India is internally heterogeneous (see Section 4, per-South-state classifier).

Q8. Is the model adaptable to new cities / a new dataset?

Yes. Adding a new city is two lines in `src/regional.py` (one in `CITY_CODE_MAP`, one in `CITY_CENTROIDS`). Any rider ID with an unknown prefix falls back to a lat/lon bounding-box classifier — no code change needed. The full pipeline is in `notebooks/_run_regional.py`, runs end-to-end in ~3 minutes on a CPU.

Q9. How do you defend the 'all 11 hypothesis tests $p > 0.05$ ' against a finding-bias accusation?

We pre-registered the tests in the developer plan PDF before running them (`reports/dev_plan_regional.pdf`, committed to git before the runner was executed). We report all 11, not a cherry-picked subset. We then

went LOOKING for hidden patterns with six more stress tests, found one (distance_km), and report it honestly with its limitation (Fold-4 anomaly).

Q10. What would beat your model?

Real-time traffic feeds, restaurant-id features (rolling prep-time history), zonal demographic data, a longer date window covering Onam/Pongal/Diwali, and a temporally held-out test set. Each is listed in the limitations section of the final report. None of these are in the dataset we were given.

6. Adaptability (judging criterion #2)

Three concrete senses in which this work transfers cleanly:

Sense	Mechanism	Code path
A new city arrives	Add one row to CITY_CODE_MAP and CITY_CENTROIDS	IS/region/region1350 Coordinate-cross-check
An unknown city code arrives	lat/lon bounding-box fallback classifies South (8°-19°N, 73°E) or North (18°-25°N, 68-69°E) without	src/region.py Line 1351
A new dataset arrives	load_clean() handles the same column dialect; build_model() takes in prod/development/y matrix; _run_re	notebooks/_prod/region1350
A new region definition (e.g. edit SOUTH_STATES)	/regional.py (one line). Re-run the pipeline	src/region.py 55 metrics, figures, and the dec
Productionising for live inference	joblib booster(model_file=...) loads in <100 ms. Cold-start to first prediction classifying v2 is clean	src/region.py 1352
Different ML framework	We have a CatBoost cross-check; switching to XGBoost is as simple as editing src/regional.py	src/regional.py 55

Adaptability headline. Every artefact (JSON metrics, figures, deck, PDFs) is built from the live regional_findings.json — re-running the pipeline on new data automatically refreshes every output downstream. Zero copy-paste of numbers between code and deck.

7. Self-audit against the 40-mark rubric

Criterion	What we did	Self-score
1. Model + technical rigor (10)	3 trained models, 5-fold StratifiedKFold (seed 42) + GroupKFold	9/10, early stopping, class_weight
2. Model adaptability (10)	City code map and lat/lon fallback handle unknown inputs without code change. Pipeline is JSON-	8/10
3. Understanding (10)	Two PDFs (final_report_regional, this guide) plus dev_plan_regional	9/10 Every term defined. Q&A che
4. Presentation (10)	10-slide pitch deck (pitch_regional_630pm.pdf) following the rule of 6 structure. Real numbers	9/10
TOTAL		35 / 40

Repo: github.com/TheClazer/Zomato-delivery-estimation · **Final commit:** see git log -1 --oneline before pitching · **This file generated by:** notebooks/_build_understanding_guide.py from live JSON.